



Wrocław University
of Science and Technology

OPTIMIZATION METHODS
PROJECT GROUP 1

Feature Selection

Main Project
— Project's Report —

Student

Sczymon CICHY
Student ID: XXX

Maxime LUCE
Student ID: 294620

Teacher

Konrad GMYREK
Marcin KOMARNICKI

Academic year 2025/2026 – May 15, 2026

Contents

1	The Feature Selection Problem	3
1.1	General overview	3
1.2	Specifics	3
1.3	Objectives	3
1.4	Constraints	3
1.5	Chosen problem representation	3
1.6	Solved example	3
2	Theoretical review	3
2.1	Solving an Optimization problem : Exploration and Exploitation	3
2.2	Differential Evolution	4
2.3	TS/SA	4
2.4	PSO	4
2.5	Heavy Model	4
2.6	Light model	4
3	Implementation and protocol	4
3.1	Description of the protocol	4
3.1.1	Hardware description and environment	4
3.1.2	Stop condition	5
3.1.3	Measuring computation time	5
3.2	Main structure	6
3.3	Unit tests	8
4	Finding the optimal parameters	8
4.1	Description of the different cases	8
4.1.1	Evolutionary Algorithm	9
4.1.2	Simulated Annealing	11
4.2	Running experiment	12
4.2.1	Evolutionary Algorithm	12
4.2.2	Simulated Annealing	23
4.3	Summary of the parameters	26



5 Solving the permutation flowshop scheduling problem	26
5.1 Comparison of all methods on every instances	26
5.2 Convergence of Evolutionary Algorithm	26
5.3 Best sequences found	26
Conclusion	27
References	27
Appendixes	28
List of Tables	28
List of Figures	29
List of Algorithms	29
List of Listings	29

Introduction

This document aims to present the main results of the project on *Feature Selection* (FS), which is an optimization problem.

We are indeed considering a set of variables and we aim to select the most relevant in a specific subset. This feature selection can be very useful to reduce data processing, to detect the core characteristics of a system or to train efficiently a model.

Present the common way to solve it

Present the example (data)

We will start by presenting the PFSP and giving a solved example in order to get familiar with the problem 1. Then, we will provide some theoretical review on formalizing an optimization problem and on methods to solve it. We will also present our implementation and protocol for our numerical analysis 3. Finally, we will present our method to determine the best set of parameters for our algorithms 4 and, to finish, we will present our solutions to the PFSP 5.

1 The Feature Selection Problem

1.1 General overview

1.2 Specifics

1.3 Objectives

1.4 Constraints

1.5 Chosen problem representation

1.6 Solved example

2 Theoretical review

2.1 Solving an Optimization problem : Exploration and Exploitation

When it comes to find an optimal solution to a given problem, we can first try to describe well our problem. They are two main categories of problem : *combinatorial* and *topological*. In optimization, a *topological* problem seeks the best continuous distribution of material within a physical space (such as designing the most efficient shape for a mechanical component), whereas a *combinatorial* problem involves finding the optimal discrete arrangement from a finite set of possibilities (such as finding the best sequence of tasks in the PFSP).

A general n –dimensional optimization problem is described by an objective function f

$$f : \mathcal{D}_f \subset \mathbb{R}^n \rightarrow \mathbb{R}, \quad (1)$$



where $\mathcal{D}_f$ is the solution search space. Our objective is therefore to find x^* which can be the maximum or the minimum of f , let's assume we are looking for a maximum (even though it means changing f to $-f$).

$$x^* = \arg \max_{x \in \mathcal{D}_x \subset \mathcal{D}_f} f(x), \quad (2)$$

where $\mathcal{D}_x$ is the feasible set.

In the permutation flowshop scheduling problem, the objective function is defined (??) and we can observe that finding the maximum of this function is really hard because this function is not linear, nor convex. Therefore, as it is a NP-hard problem, we cannot use algorithms to solve this problem. Instead, we will use several methods to look for the best solution possible in the feasible set. We will now present these approaches: the random search ??, the greedy algorithm ??, the evolutionary algorithms ?? and the simulated annealing approach ??.

2.2 Differential Evolution

2.3 TS/SA

2.4 PSO

2.5 Heavy Model

2.6 Light model

3 Implementation and protocol

3.1 Description of the protocol

This section will describe the numerical protocol that we apply to compare efficiently and in a fair play way all our methods to solve the PFSP problem. First, we will describe the environment and the hardware used 3.1.1. Then, we will precise the stop condition that we used 3.1.2 and how we measured computation time 3.1.3.

3.1.1 Hardware description and environment

Computer used to run the tests All numerical experiments were executed locally on a single machine to ensure a fair and consistent evaluation environment. The hardware configuration of the computer used for these tests is detailed as follows:



Component	Specifications
Processor (CPU)	Intel® Core™ i5-1035G1 @ 1.00 GHz (Base: 1.20 GHz). 4 Physical Cores, 8 Logical Processors. 6.0 MB L3 Cache.
Memory (RAM)	12.0 GB SODIMM operating at 3200 MT/s (Dual-slot configuration).
Operating System	Ubuntu

Table 1: Hardware specifications of the machine used for computation.

Furthermore, to maintain the integrity of the computation time measurements, background tasks were minimized during the execution of the scripts.

Environment and resources allocation UBUNTU allows you to allocate a certain number of cores to a specific task, that's why we allocate a specific core for every experiment run. To do so, we use the following:

```
1 # force the script to run only on core 1
2 taskset -c 1 python app.main
```

Listing 1: Allocating core 1 to run the experiment

We also control the number of core use by Python by adding the code **Listing 2** in our `main` file.

```
import os
os.environ["OMP_NUM_THREADS"] = "1" # OpenMP
os.environ["OPENBLAS_NUM_THREADS"] = "1" # OpenBLAS
os.environ["MKL_NUM_THREADS"] = "1" # MKL
os.environ["VECLIB_MAXIMUM_THREADS"] = "1" # Accelerate
os.environ["NUMEXPR_NUM_THREADS"] = "1" # NumExpr
```

Listing 2: Control the number of cores used in the `main` file of our application

3.1.2 Stop condition

To ensure a strictly fair and unbiased comparison between the different stochastic algorithms (Random Search, Evolutionary Algorithm, and Simulated Annealing), we established a common computational budget rather than relying on a time-based limit. The stop condition was strictly defined by a maximum Number of Function Evaluations (NFE) set to 10 000.

The only exception to this rule is the Greedy algorithm. Being a deterministic constructive heuristic rather than a stochastic search method, it does not iterate over complete solutions. Instead, it naturally terminates as soon as the single optimal local schedule is fully built, resulting in an NFE of exactly 1.

3.1.3 Measuring computation time

It is also useful to accurately measure the real-world execution time in order to evaluate the practical efficiency of each algorithm.

To measure the elapsed time in our Python implementation, we use the `timeit` module instead of the most common `time` as `timeit` is much more accurate [1]. As widely documented in Python



benchmarking standards, `time.time()` measures the real-world wall-clock time and is susceptible to system clock updates, which can lead to inaccuracies during very short executions.

3.2 Main structure

According to the pseudo-code presented in 2, we have implemented these methods using the Object-oriented programming and the structure is therefore represented through a class diagram **Figure 1**.

All the code and the files of the project is available here: <https://github.com/MaximeLuce/PFSP>.

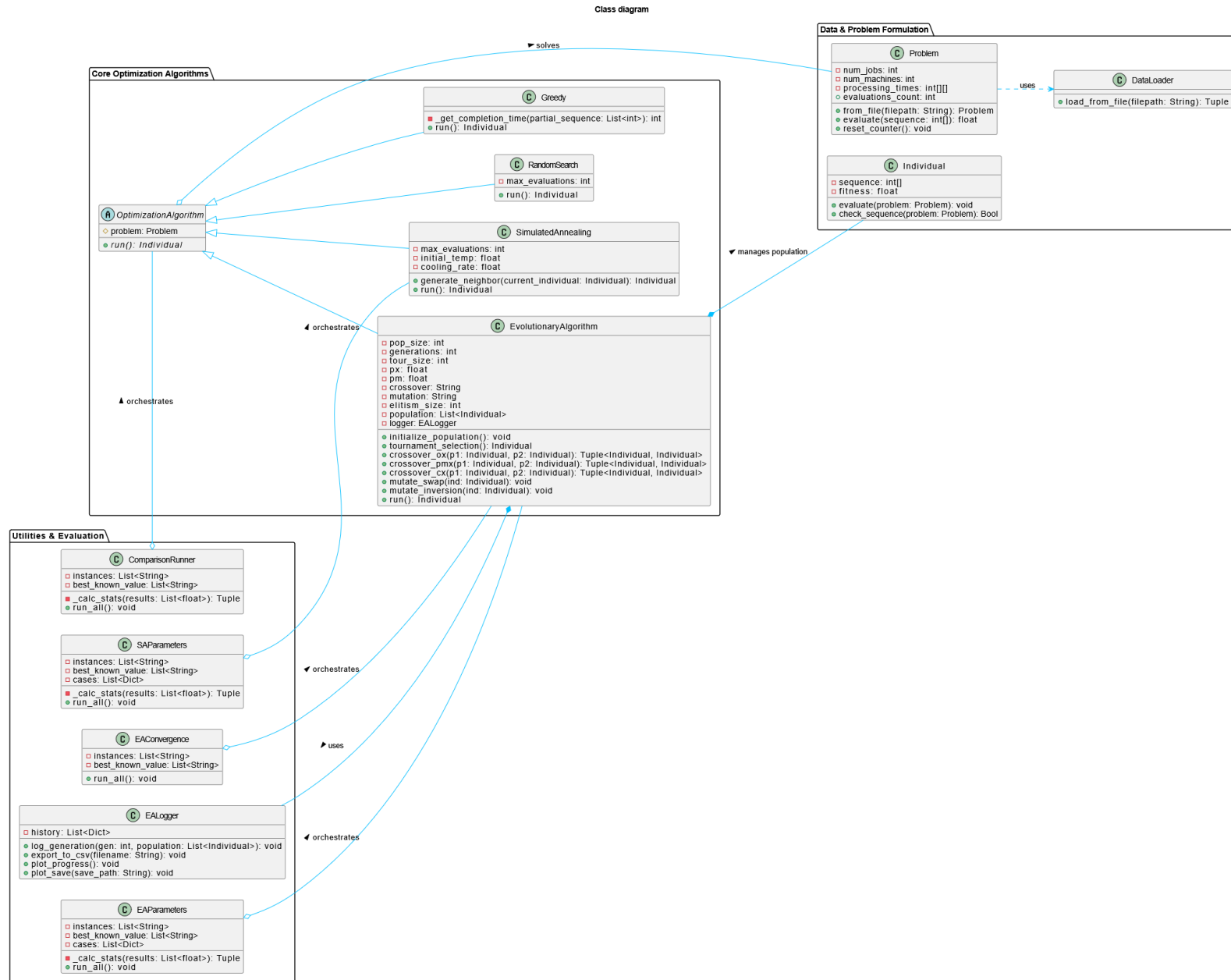


Figure 1: Class diagram of the application implemented.

3.3 Unit tests

Before running all numerical experiments, we implemented unit tests in order to verify critical method or method of algorithm such as the example [Listing 3](#): testing with example in section 2 of OX, PMX, CX, swap, inversion, tournament and problem evaluation.

```
from app.Problem.Problem import Problem

# TEST

if __name__ == "__main__":
    M = [[30, 29, 19, 50],
          [39, 69, 6, 73],
          [3, 86, 58, 60],
          [72, 74, 51, 15],
          [24, 47, 22, 90]]

    p = Problem(4,5,M)

    print("--- Display test ---")
    print(p)

    print("--- Test on known sequence ---")
    result = p.evaluate([0, 2, 3, 1])
    if result == 1170:
        print(f'Unit test successful, for the sequence [1, 3, 4, 2], we get {result}')
    else:
        print(f"Unit test failed, for the sequence [1, 3, 4, 2], we get {result}')
```

Listing 3: Example of unit test for the evaluation function of a given problem

4 Finding the optimal parameters

As described in [Table ??](#), we will use two methods that require parameters : the Evolutionary Algorithm (EA) and the Simulated Annealing (SA). To compare all these methods, we need to find first the optimal parameters for EA and SA.

First, we present all the cases used to determine the best set of parameters for these two methods 4.1. Then, we run experiment for each case 4.2 and we summarize all the best set of parameters in 4.3.

4.1 Description of the different cases

We will first present the cases in order to determine the best set of parameters for the evolutionary algorithm : the population/generation ratio, crossover probability, mutation probability, tour size, mutation type, crossover type, elitism size 4.1.1.



Then, we will present the cases for the Simulated Annealing including initial temperature and cooling rate parameters 4.1.2.

4.1.1 Evolutionary Algorithm

First, we explore the impact of changing the ratio population/generation size. But, in order to run fair test, we fix the total number of iteration, which is the product of the population and the generation size (here $100 \times 100 = 10\,000$). All other settings remain at their default values.

These cases (Table 2) are designed to try to find a trade-off between, on the one-hand *exploration* (large population and few generation) and *exploitation* (small population and many generations).

As mentioned in the protocol in section 3, we also use average our results over 10 runs.

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
1	100	100	0.7	0.1	5	Swap	OX	0	10
2	20	500	0.7	0.1	5	Swap	OX	0	10
3	1000	10	0.7	0.1	5	Swap	OX	0	10
4	10000	1	0.7	0.1	5	Swap	OX	0	10

Table 2: Description of the parameters of the cases used to determine the best population/generation ratio.

We assume then that we get the best ratio between population and generation size, denoted by a^* and b^* , respectively. Thus, we can study the impact of the crossover probability on the score (Table 3). We also include inside medium condition ($P_x = 0.3$) boundary conditions such as $P_x = 0$ (no crossover) and forced crossover $P_x = 1$.

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
5	a^*	b^*	0.0	0.1	5	Swap	OX	0	10
6	a^*	b^*	0.3	0.1	5	Swap	OX	0	10
7	a^*	b^*	0.7	0.1	5	Swap	OX	0	10
8	a^*	b^*	1.0	0.1	5	Swap	OX	0	10

Table 3: Description of the parameters of the cases used to determine the best crossover probability.

We note c^* the best crossover probability found and we now focus our analysis to the mutation probability P_m . In the same way than the previous parameter, we explore boundary conditions ($P_m = 0$ and $P_m = 1$) as well as a very low rate ($P_m = 0.01$) suitable for swap operation [2] and the default value ($P_m = 0.1$).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
9	a^*	b^*	c^*	0.0	5	Swap	OX	0	10
10	a^*	b^*	c^*	0.01	5	Swap	OX	0	10
11	a^*	b^*	c^*	0.1	5	Swap	OX	0	10
12	a^*	b^*	c^*	1.0	5	Swap	OX	0	10

Table 4: Description of the parameters of the cases used to determine the best mutation probability.

We now note d^* the best mutation probability and we continue our exploration with the size of the tournament (parameter "Tour") (Table 5). We use the default value (5) and as this parameter describe the selection pressure, we explore a bigger value (20) and the boundary conditions : no selection pressure (1) and maximum pressure with the tournament size equal to the population size (a^*).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
13	a^*	b^*	c^*	d^*	1	Swap	OX	0	10
14	a^*	b^*	c^*	d^*	5	Swap	OX	0	10
15	a^*	b^*	c^*	d^*	20	Swap	OX	0	10
16	a^*	b^*	c^*	d^*	a^*	Swap	OX	0	10

Table 5: Description of the parameters of the cases used to determine the best tour size.

Let's assume that we get the best tournament size, that we note e^* . We now compare the scores obtained with the two types of mutation : swap and inversion (Table 6).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
17	a^*	b^*	c^*	d^*	e^*	Swap	OX	0	10
18	a^*	b^*	c^*	d^*	e^*	Inversion	OX	0	10

Table 6: Description of the parameters of the cases used to determine the best mutation type. Comparison between swap and inversion mutation.

We then do the same approach with the crossover types: Ordered Crossover (OX), Partially Matched Crossover (PMX) and the Cycle Crossover (CX) (Table 7).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
19	a^*	b^*	c^*	d^*	e^*	f^*	OX	0	10
20	a^*	b^*	c^*	d^*	e^*	f^*	PMX	0	10
21	a^*	b^*	c^*	d^*	e^*	f^*	CX	0	10

Table 7: Description of the parameters of the cases used to determine the best crossover type.

Finally, we study the impact of the presence of elitism which was absent for all our first cases. Therefore, we compare the situation without elitism (0), and with just one and finally five (Table 8).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
22	a^*	b^*	c^*	d^*	e^*	f^*	g^*	0	10
23	a^*	b^*	c^*	d^*	e^*	f^*	g^*	1	10
24	a^*	b^*	c^*	d^*	e^*	f^*	g^*	5	10

Table 8: Description of the parameters of the cases used to determine the impact of elitism.

4.1.2 Simulated Annealing

We now present the same approach for the simulated annealing method with only two parameters : the initial temperature and the cooling rate. We use common value (see section ??) and try slightly different Initial Temperature (Table 9) in order to find an equilibrium between early exploration and convergence speed.

Case n°	Initial Temperature	Cooling Rate	Run per algo
25	100	0.99	10
26	1000	0.99	10
27	10000	0.99	10

Table 9: Description of the parameters used to determine the best initial temperature for Simulated Annealing.

Let's assume that we have the best initial temperature that we note T^* . We then run the method with a cooling rate closer to one (Table 10).

Case n°	Initial Temperature	Cooling Rate	Run per algo
28	T^*	0.90	10
29	T^*	0.99	10
30	T^*	0.999	10
31	T^*	0.9999	10

Table 10: Description of the parameters used to determine the best cooling rate for Simulated Annealing.

4.2 Running experiment

We will start by finding an optimal set of parameters for the evolutionary algorithm 4.2.1 and then for the simulated annealing 4.2.2.

For every table of results : each instance is described by its number of tasks and machines as $J \times M$; the color green is used to highlight the best score and the best average score for each instance and the color red for the worst ; Evolutionary Alg. [10x] indicates ten iterations of the evolutionary algorithm and best* (best* indicates the best value from each 10 runs), worst, average and standard deviation (std), computation time and Number of Function Evaluations (NFE) are provided for each case and finally the best known value is indicated.

For each parameters, we present the results from each instance of the same level of difficulty in a separate table.

4.2.1 Evolutionary Algorithm

Best Ratio generation/population size We start by determining the best ratio generation/population size. We present in different table the result from instances of different difficulties: the basic instances in Table 11, the medium instance in Table 12 and the hard instance in Table 13.

For the basic instance, we can see that the best case is the number one as it has the best average score for every cases. The best score for the instance 2 is obtained with case 2 but is very close to the case 1 and, as the process uses randomness, this difference is not relevant. We also can see that all cases have the same computation time but the case 4 is slightly faster. This case 4 has indeed a huge population size but only one generation, it is therefore like a random search which decrease the average score but decrease also the computation time.

		Evolutionary Alg. [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	1	2	3	4	1	2	3	4	1	2	3	4
Score	best*	14222.0	14254.0	14285.0	15352.0	21102.0	21094.0	21417.0	22829.0	33963.0	34113.0	34172.0	35670.0
	worst	14647.0	14569.0	14605.0	15628.0	22057.0	22304.0	21935.0	23243.0	34517.0	35238.0	34813.0	36238.0
	avg	14366.2	14374.9	14410.7	15482.9	21689.0	21742.2	21725.9	23075.6	34304.3	34642.4	34585.7	36036.2
	std	125.35	106.58	99.63	81.98	314.25	341.53	196.77	130.07	182.33	353.26	191.34	187.23
Time	second (s)	10.887	10.731	11.085	8.956	19.405	19.209	19.055	17.305	36.425	36.045	36.032	33.833
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 11: Results for each case and each basic instance to get the best ratio generation/population size for the evolutionary algorithm.

We get the same results for the medium instances with the best score and the best average value for the case 1 and the worst for the case 4 but with a smaller computation time.

		Evolutionary Alg. [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	1	2	3	4	1	2	3	4
Score	best*	324127.0	324276.0	342726.0	358835.0	398937.0	402332.0	416916.0	435885.0
	worst	331236.0	335865.0	346552.0	364417.0	408207.0	411199.0	424099.0	441092.0
	avg	327296.7	330382.8	344602.9	361473.6	404436.0	406637.8	421884.1	439011.4
	std	2411.51	3262.17	1477.53	1905.43	2741.5	3743.87	2058.67	1452.86
Time	second (s)	93.392	92.168	90.77	83.388	179.086	183.177	177.992	169.066
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 12: Results for each case and each medium instance to get the best ratio generation/population size for the evolutionary algorithm.

Regarding the hard case, the worst case is still the case 4 but the case with the best average value is now case 2. Indeed, this case with a small population (20) but a lot of generations (500) has a bigger exploration space than the case 1 with only 100 generations. Therefore, as the search space of the hard instance is bigger, the case 2 is better. However, we can notice that the difference between the average value of case 1 and 2 is only 0.016%.

Finally, we can conclude that the case 1 is the best overall (except for the hard instance but the difference is very minor). Therefore, we adopt the parameters of 100 individuals in the population and 100 generations, so $a^* = 100$ and $b^* = 100$.

		Evolutionary Alg. [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	1	2	3	4
Score	best*	7500875.0	7490765.0	7746411.0	7888137.0
	worst	7608577.0	7576741.0	7786773.0	7943570.0
	avg	7536436.2	7535204.7	7765615.4	7915677.5
	std	33531.22	28554.9	12073.46	19238.63
Time	second (s)	965.183	970.792	959.711	839.728
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 13: Results for each case on the hard instance to get the best ratio generation/population size for the evolutionary algorithm.

Best Crossover probability Once we have $a^* = 100$ and $b^* = 100$, we can proceed the cases 5 to 8 in order to determine the best crossover probability.

For the basic instances, we can see that the case with the best average value is always the case 8 and the worst (on average) is the case 5 (Table 14). Indeed the case 5 corresponds to a probability $P_x = 0$ so there is no more crossover. Therefore, the method just copy individuals with just a mutation so the algorithm loses its capability to explore a huge space domain and the algorithm is just exploring randomly in the local search space. Thus, the mutation alone is not able to converge the evolutionary algorithm. We also observe that the best score for instance 2 and 8 are obtained with the case 7 but the value is very close to the case 8 and, as the method is random, the difference is not relevant.

		Evolutionary Alg. [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	5	6	7	8	5	6	7	8	5	6	7	8
Score	best*	14183.0	14198.0	14130.0	14096.0	21419.0	21412.0	21001.0	21030.0	34009.0	34127.0	33830.0	33983.0
	worst	14668.0	14559.0	14387.0	14406.0	22395.0	22010.0	21952.0	21747.0	35228.0	34665.0	34789.0	34899.0
	avg	14378.2	14348.6	14270.2	14197.8	21925.9	21775.4	21513.1	21350.6	34644.5	34434.9	34510.0	34282.4
	std	171.85	100.73	86.03	91.4	334.96	185.14	268.22	204.99	316.67	162.93	289.64	264.75
Time	second (s)	10.772	10.724	10.639	10.683	19.758	19.641	19.197	18.985	35.865	35.823	35.749	35.7
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 14: Results for each case and each basic instance to get the best crossover probability for the evolutionary algorithm.

We observe the same trend for the medium instances with the worst average case (5) and the best average case (8) (Table 15). The case 8 correspond to a crossover probability equal to 1 so there is a crossover at each step so the evolutionary algorithm is constantly changing. As the other default parameters are a good pressure selection with a tour size of 5 and an OX-crossover, this crossover probability helps the algorithm to find local optimum faster. We can also notice that for this case, the standard deviation is much smaller (46.81% for instance 1, 38.9% for instance 2, 27% for instance 4 for example) which leads us to conclude that convergence is happening more quickly.

		Evolutionary Alg. [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	5	6	7	8	5	6	7	8
Score	best*	331935.0	326407.0	326543.0	322743.0	408184.0	405759.0	401258.0	397746.0
	worst	346741.0	336932.0	334801.0	333509.0	417669.0	413777.0	408353.0	406913.0
	avg	338030.3	332789.2	331122.8	327199.8	413156.8	408785.9	405320.2	401610.5
	std	4214.69	3430.35	2507.97	3076.81	2854.14	2682.9	2583.73	3054.56
Time	second (s)	89.903	90.907	92.264	93.806	173.785	176.184	177.271	180.553
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 15: Results for each case and each medium instance to get the best crossover probability for the evolutionary algorithm.

Regarding the hard instance, we get the same result and a standard deviation for the case 8 35.17% smaller than the case 5. We can also observe that the computation is bigger for case 8 than case 5 as it takes time to run the crossover (there is no more crossover for case 5).

Finally, we can conclude that the best overall case is the case 8 with a crossover probability of $P_x = 1$ so we get $c^* = 1$.

		Evolutionary Alg. [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	5	6	7	8
Score	best*	7633865.0	7558436.0	7495775.0	7440933.0
	worst	7753107.0	7633119.0	7559650.0	7509208.0
	avg	7706413.5	7585329.5	7518667.2	7476693.9
	std	32814.73	27160.69	19751.69	21272.84
Time	second (s)	856.528	910.855	986.947	1035.307
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 16: Results for each case on the hard instance to get the best crossover probability for the evolutionary algorithm.

Best Mutation Probability As we get $c^* = 1$, we can now run the experiments to determine the best mutation probability (cases 9 to 12).

We can see that the best average value is for the case 12 for the basic instances (Table 17), which correspond to a mutation probability of one. This result is interesting because it should mean that all the sequence is mutated so the information is destroyed. However, this is not the case as we have implemented the mutation on the whole individual and not on each gene (see Listing 4). Therefore, even with a mutation probability of 1, we only change an unique swap so this huge probability is in fact beneficial for the exploration.

```

if random.random() < self.pm:
    idx1, idx2 = random.sample(range(len(individual.sequence)), 2)
    individual.sequence[idx1], individual.sequence[idx2] = ...

```

Listing 4: Implementation of the mutation

		Evolutionary Alg. [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	9	10	11	12	9	10	11	12	9	10	11	12
Score	best*	14203.0	14205.0	14095.0	14158.0	21129.0	21228.0	21459.0	21175.0	34198.0	34025.0	34014.0	34127.0
	worst	14943.0	14738.0	14465.0	14448.0	21950.0	21975.0	22071.0	21934.0	35041.0	34675.0	34585.0	34462.0
	avg	14460.3	14382.0	14295.0	14281.1	21555.5	21629.7	21736.1	21476.1	34615.4	34373.9	34334.3	34320.9
	std	219.84	163.85	120.92	88.7	275.18	230.69	220.81	211.97	261.74	239.99	174.83	142.37
Time	second (s)	10.731	10.643	10.634	11.124	20.046	19.258	19.038	19.182	35.638	35.658	35.682	35.936
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 17: Results for each case and each basic instance to get the best mutation probability for the evolutionary algorithm.

We observe the same results for the medium instances Table 18, the worst case is still the case 9 where the mutation probability is 0. In this case, indeed, there is no more mutation so we cannot create again a lost gene. Therefore, the algorithm is converging prematurely and that proves that mutation is absolutely necessary to maintain enough diversity in the population.

		Evolutionary Alg. [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	9	10	11	12	9	10	11	12
Score	best*	329304.0	326378.0	321181.0	322710.0	407525.0	400084.0	397951.0	398659.0
	worst	341982.0	335778.0	330675.0	330083.0	411538.0	410820.0	406257.0	405660.0
	avg	334502.0	331145.4	326455.5	326618.8	409689.2	406938.4	402739.6	401484.8
	std	4057.82	3223.72	3115.71	2270.6	1218.14	3436.72	2651.99	2604.52
Time	second (s)	93.16	91.567	91.614	91.862	175.273	174.932	175.036	175.408
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 18: Results for each case and each medium instance to get the best mutation probability for the evolutionary algorithm.

We observe the same best and worst average case for the hard instance (Table 19). We finally can notice that results from case 11 ($P_m = 0.1$) and 12 ($P_m = 1$) are very close and the case 11 is even better than case 12 in the instance 4. However, as the case 12 promotes exploration, it is even better when the instance is hard.

We can therefore conclude that the best overall is the case 12 with a mutation probability of $P_m = 1$, thus $d^* = 1$.

		Evolutionary Alg. [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	9	10	11	12
Score	best*	7478639.0	7528530.0	7413854.0	7320732.0
	worst	7627611.0	7583571.0	7545384.0	7465969.0
	avg	7575187.3	7558861.3	7464309.6	7405428.6
	std	41445.27	21023.83	43909.96	39549.72
Time	second (s)	1060.226	1070.248	1069.685	1070.132
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 19: Results for each case on the hard instance to get the best mutation probability for the evolutionary algorithm.

Best Tour Size As we just get $d^* = 1$, we are now able to run the tests on the tour size (cases 13 to 16).

First, we can notice that the overall worst case is the case 13 with tour size of 1 because there is therefore no selection pressure so the parents are chosen without any regard on the quality of the individual. Thus, this case is more likely the random search.

We can observe on the basic instances that the extreme selection of a tour size of 100 (case 16) does not provide as good solution as case 15 (with a tour size of 20) (Table 20).

		Evolutionary Alg. [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	13	14	15	16	13	14	15	16	13	14	15	16
Score	best*	15293.0	14231.0	14054.0	14083.0	22778.0	21320.0	20971.0	21159.0	35672.0	34261.0	33770.0	34040.0
	worst	15608.0	14404.0	14255.0	14373.0	23373.0	22000.0	21581.0	22065.0	36364.0	34606.0	34342.0	34489.0
	avg	15402.3	14289.5	14120.9	14199.7	23119.4	21582.5	21248.4	21596.5	36065.5	34440.7	34119.3	34301.9
	std	122.88	60.49	64.6	93.74	231.07	268.92	183.91	305.57	242.46	106.08	180.64	167.16
Time	second (s)	10.646	10.779	11.602	13.6	19.812	20.343	21.039	22.73	37.671	37.792	38.333	40.113
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 20: Results for each case and each basic instance to get the best tour size for the evolutionary algorithm.

On the medium instances, the worst case is still the case 13 but the best case becomes the case 16 as this pressure is interacting with the mutation probability (Table 21). However, the case 15 provides very close results (0.6% for the instance 4 for example).

		Evolutionary Alg. [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	13	14	15	16	13	14	15	16
Score	best*	360831.0	322772.0	319697.0	315635.0	435493.0	397925.0	391204.0	388737.0
	worst	364669.0	328417.0	323292.0	321219.0	442487.0	404076.0	396854.0	398003.0
	avg	362581.4	325470.8	321665.9	319593.4	439164.3	401260.9	394464.9	393110.1
	std	1313.51	1706.17	1102.11	1717.19	2100.67	2326.47	1641.03	2706.11
Time	second (s)	95.031	97.485	98.495	100.354	186.515	186.394	184.712	184.812
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 21: Results for each case and each medium instance to get the best tour size for the evolutionary algorithm.

On the hard instance (Table 22), the best overall case is still the case with an extreme pressure selection (case 16). As this case is not able to provide good results on the basic instances and as the case 15 provides better solution on these instances and still find close solutions for medium and hard instances, we can conclude that the best overall tour size is 20 (case 15). Moreover, this is more than the default value of 5 because we had already a lot of exploration (as $P_x = 1$ and $P_m = 1$) so we need a pressure selection to counterbalance this huge exploration. Thus, we get $c^* = 20$.

		Evolutionary Alg. [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	13	14	15	16
Score	best*	7865718.0	7377624.0	7320370.0	7316248.0
	worst	7963803.0	7451755.0	7380543.0	7372040.0
	avg	7919021.9	7411693.7	7357453.1	7350482.0
	std	36021.12	25345.13	20160.08	21873.88
Time	second (s)	1045.47	1064.819	1073.462	1074.66
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 22: Results for each case on the hard instance to get the best tour size for the evolutionary algorithm.

Best Mutation Type We will now present the results in order to select either the swap or the inversion mutation. The results of these mutation are very close for the basic instances but the swap is on average better (Table 23). For the instance 3, by chance the inversion get a better best score.

		Evolutionary Alg. [10x]					
		Inst. 1 (20x5)		Inst. 2 (20x10)		Inst. 3 (20x20)	
Data	Case n° Stat.	17	18	17	18	17	18
Score	best*	14059.0	14085.0	21011.0	21012.0	33995.0	33953.0
	worst	14232.0	14222.0	21663.0	22190.0	34354.0	34555.0
	avg	14114.0	14139.2	21317.4	21514.5	34132.3	34243.0
	std	61.48	51.08	209.62	438.97	104.5	228.18
Time	second (s)	11.522	11.625	20.522	21.091	38.49	38.453
NFE	number	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	20911	20911	33623	33623

Table 23: Results for each case and each basic instance to get the best mutation type for the evolutionary algorithm.

The swap mutation is still the best mutation type for the medium instances but we can observe that the difference between these two methods is greater (Table 24).

		Evolutionary Alg. [10x]			
		Instance 4 (100x10)		Instance 5 (100x20)	
Data	Case n° Stat.	17	18	17	18
Score	best*	317347.0	325925.0	391503.0	406077.0
	worst	323744.0	338217.0	399328.0	417778.0
	avg	320827.7	331761.3	395960.8	412763.4
	std	1803.07	3917.07	2911.22	3922.15
Time	second (s)	97.18	97.127	178.997	177.733
NFE	number	10000	10000	10000	10000
Best Known Value		300566	300566	367267	367267

Table 24: Results for each case and each medium instance to get the best mutation type for the evolutionary algorithm.

When it comes to the hard instance (Table 25), the difference is now of approximately 240 000 so the swap is definitively better for the hard instance. Therefore, we can conclude that the best mutation type overall is the swap so we get $f^* = swap$.

		Evolutionary Alg. [10x]	
		Instance 6 (500x20)	
Data	Case n° Stat.	17	18
Score	best*	7315598.0	7529857.0
	worst	7388985.0	7656086.0
	avg	7355471.4	7596958.2
	std	22598.95	37357.41
Time	second (s)	1048.25	1047.768
NFE	number	10000	10000
Best Known Value		6697476	6697476

Table 25: Results for each case on the hard instance to get the best mutation type for the evolutionary algorithm.

Best Crossover Type We will now present the results for the choice of the crossover type.

For the basic instances, the crossover is the best mutation on average but all mutation type have similar value (Table 26). We can notice that we reach the best known value with the OX mutation for the instance 3 (color yellow).

		Evolutionary Alg. [10x]								
		Inst. 1 (20x5)			Inst. 2 (20x10)			Inst. 3 (20x20)		
Data	Case n° Stat.	19	20	21	19	20	21	19	20	21
Score	best*	14062.0	14046.0	14034.0	21023.0	20958.0	21117.0	33623.0	33812.0	33887.0
	worst	14251.0	14287.0	14258.0	21609.0	21884.0	21686.0	34453.0	34381.0	34415.0
	avg	14132.7	14160.6	14146.3	21228.9	21325.7	21399.6	34036.5	34126.5	34097.0
	std	69.68	70.15	74.73	184.67	243.52	183.82	211.52	178.98	161.31
Time	second (s)	11.409	11.171	10.835	20.728	20.675	19.473	36.724	37.309	36.87
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	20911	20911	20911	33623	33623	33623

Table 26: Results for each case and each basic instance to get the best crossover type for the evolutionary algorithm. The color yellow indicates that we reach the best known value for the case 19 on the instance 3.

On medium instances, we can observe that the CX mutation is the worst (Table 27). This mutation preserves indeed the absolute position of an element but not the relative position. However, in the PFSP, it is the relative position that matters the most and that is the reason why the CX mutation is not optimal for this problem.

		Evolutionary Alg. [10x]					
		Instance 4 (100x10)			Instance 5 (100x20)		
Data	Case n° Stat.	19	20	21	19	20	21
Score	best*	316081.0	314947.0	318773.0	389043.0	390247.0	393423.0
	worst	323333.0	322210.0	323968.0	399255.0	399593.0	398363.0
	avg	320083.2	318714.1	321398.7	394244.6	394376.4	396291.5
	std	2140.83	2103.99	1675.99	2613.91	2868.01	1556.02
Time	second (s)	94.265	88.484	87.536	180.122	173.232	172.941
NFE	number	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	367267	367267	367267

Table 27: Results for each case and each medium instance to get the best crossover type for the evolutionary algorithm.

On the medium instances and on the hard instance, we can observe that both OX and PMX obtained similar good results. However, we can see that the PMX is much faster than the OX, especially on the hard instance (Table 27) with a difference of 16.96% for the instance 6. That is why, even if both crossover have the same results (OX slightly better for the medium and basics instances and PMX slightly better for the hard instance), we can finally conclude that the PMX crossover is the best overall. Thus, we have $g^* = \text{PMX}$.

		Evolutionary Alg. [10x]		
		Instance 6 (500x20)		
Data	Case n° Stat.	19	20	21
Score	best*	7326669.0	7295784.0	7351373.0
	worst	7394134.0	7347446.0	7449460.0
	avg	7352788.6	7323820.3	7397009.3
	std	19627.19	15456.29	30614.76
Time	second (s)	1038.401	868.282	853.303
NFE	number	10000	10000	10000
Best Known Value		6697476	6697476	6697476

Table 28: Results for each case on the hard instance to get the best crossover type for the evolutionary algorithm.

Impact of Elitism We have now all our final parameters and we can now study the impact of elitism. First, we can observe that the NFE for the case 24 (elitism of 5) is not 10 000 but 9 604 and this difference comes from our implementation of elitism (Table 29). As we copy 5 best individuals from the previous generation, we just have to create 95 new children to fill our generation so we loop 48 times (because we create children by pairs so $48 \times 2 = 96$ so we have in total $100 + (99 \times 96) = 9604$ evaluations in total.

Furthermore, we can observe that the case without elitism (case 22) is the best on average for the basic

instances.

		Evolutionary Alg. [10x]								
		Inst. 1 (20x5)			Inst. 2 (20x10)			Inst. 3 (20x20)		
Data	Case n° Stat.	22	23	24	22	23	24	22	23	24
Score	best*	14066.0	14060.0	14041.0	20955.0	20981.0	21081.0	33897.0	33939.0	33983.0
	worst	14247.0	14334.0	14323.0	21769.0	21979.0	21879.0	34413.0	34430.0	34544.0
	avg	14137.9	14159.8	14189.0	21226.4	21447.5	21401.8	34162.5	34218.3	34210.6
	std	56.79	87.22	85.91	211.48	348.45	262.52	151.68	155.91	168.04
Time	second (s)	11.132	11.12	10.805	20.359	20.595	18.969	36.431	36.907	35.28
NFE	number	10000	10000	9604	10000	10000	9604	10000	10000	9604
Best Known Value		14033	14033	14033	20911	20911	20911	33623	33623	33623

Table 29: Results for each case and each basic instance to study the impact of elitism for the evolutionary algorithm.

Regarding the medium instances, the case without elitism and with a little bit of elitism (case 23) are very close but the case with a lot of elitism (case 24) is the worst (Table 30).

		Evolutionary Alg. [10x]					
		Instance 4 (100x10)			Instance 5 (100x20)		
Data	Case n° Stat.	22	23	24	22	23	24
Score	best*	314503.0	315850.0	315638.0	391221.0	389343.0	389667.0
	worst	322209.0	321540.0	324126.0	396354.0	396602.0	396029.0
	avg	318695.9	318630.6	319206.7	393746.6	393634.2	392964.9
	std	2122.58	2036.45	2940.23	1883.06	2415.28	2477.69
Time	second (s)	89.431	88.455	85.575	175.416	175.712	167.002
NFE	number	10000	10000	9604	10000	10000	9604
Best Known Value		300566	300566	300566	367267	367267	367267

Table 30: Results for each case and each medium instance to study the impact of elitism for the evolutionary algorithm.

When it comes to the hard instance, the case without elitism is the best and the difference with the other cases is bigger than the medium instances. As we have a very dynamic algorithm with $P_m = 1$ and $P_x = 1$, the elitism will keep the best solution and slow the exploration dynamic so the algorithm will converge prematurely.

Before concluding, we must question the fairness of the results as the NFE is not the same for the three cases. First, we need to notice that there is just a difference of 4% between 10 000 and 9 604 so it is very likely to not change the results (especially on the instance 6 where the difference is big) because the difference is equivalent to just 4 generations, which is not enough to change significantly the results.

Thus, we can conclude that the best parameter overall is no elitism so $h^* = 0$.

		Evolutionary Alg. [10x]		
		Instance 6 (500x20)		
Data	Case n° Stat.	22	23	24
Score	best*	7303082.0	7320097.0	7323373.0
	worst	7358827.0	7384673.0	7376432.0
	avg	7329992.0	7347818.3	7353550.7
	std	19016.22	21781.09	20212.65
Time	second (s)	851.547	854.795	827.366
NFE	number	10000	10000	9604
Best Known Value		6697476	6697476	6697476

Table 31: Results for each case on the hard instance to study the impact of elitism for the evolutionary algorithm.

4.2.2 Simulated Annealing

Best Initial Temperature We will now present our results to determine the best initial temperature from 100 and 1000 to 10000. All these temperatures provide very close results on the basic instances [Table 32](#), the medium instances [Table 33](#) and the hard instance [Table 34](#).

However, we can observe on these tables that the case 26 (so an initial temperature of 1000) is the best on average on the instances 1, 3, 4 and 6 (the most difficult) and, moreover, that the standard deviation is lower on the instances 3, 4 and 6, which led us to conclude that this temperature is slightly better for our problem.

The case 25 correspond indeed to an initial temperature of 100 lower than 1000 so the algorithm will refuse faster the false solution and the case 27 correspond to 10000 which is better for the exploration phase but this phase is therefore longer and the exploitation is not enough, especially in the instance 6.

Thus, we can conclude that the initial temperature of 1000 is the best overall and we have therefore $T^* = 1000$.

		Simulated Annealing [10x]								
		Inst. 1 (20x5)			Inst. 2 (20x10)			Inst. 3 (20x20)		
Data	Case n° Stat.	25	26	27	25	26	27	25	26	27
Score	best*	14129.0	14211.0	14080.0	21189.0	21434.0	21291.0	34096.0	34088.0	34138.0
	worst	14644.0	14544.0	14697.0	22335.0	22252.0	22253.0	35603.0	34882.0	35071.0
	avg	14363.9	14331.3	14345.4	21792.7	21806.5	21773.5	34787.1	34436.0	34630.3
	std	167.81	101.78	167.89	401.44	325.76	308.15	474.31	270.61	297.21
Time	second (s)	8.504	8.462	8.536	17.052	17.628	17.708	34.079	33.488	34.009
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	20911	20911	20911	33623	33623	33623

Table 32: Results for each case and each basic instance to get the best initial temperature for the simulated annealing.



		Simulated Annealing [10x]					
		Instance 4 (100x10)			Instance 5 (100x20)		
Data	Case n° Stat.	25	26	27	25	26	27
Score	best*	313684.0	313770.0	313280.0	387497.0	387210.0	386715.0
	worst	318940.0	316809.0	319616.0	392393.0	394287.0	393934.0
	avg	316996.6	315332.2	315959.9	389913.2	390828.7	389639.7
	std	1573.81	1082.29	1769.95	1497.3	2714.8	2760.49
Time	second (s)	83.407	84.381	84.224	170.249	170.942	171.045
NFE	number	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	367267	367267	367267

Table 33: Results for each case and each medium instance to get the best initial temperature for the simulated annealing.

		Simulated Annealing [10x]		
		Instance 6 (500x20)		
Data	Case n° Stat.	25	26	27
Score	best*	7153416.0	7178187.0	7162626.0
	worst	7253803.0	7241575.0	7270542.0
	avg	7207245.8	7206331.1	7217178.8
	std	32178.82	20538.91	34666.81
Time	second (s)	838.654	840.779	837.955
NFE	number	10000	10000	10000
Best Known Value		6697476	6697476	6697476

Table 34: Results for each case on the hard instance to get the best initial temperature for the simulated annealing.

Best Cooling Rate As presented in ??, equation (??), the current temperature is given by the following:

$$T = \lambda^{\text{NFE}} T_0 \quad (3)$$

Let's compare this temperature for the different cooling rate λ . If $\lambda = 0.9$ (case 28), the temperature is near 0 after a few hundred iterations so the algorithm will process a random search after a few hundred iterations which explain why the results of this case is average.

If $\lambda = 0.99$ (case 29), the temperature is also near 0 very fast but not as fast as in the case 28 so this case is a compromise between exploitation and exploration and this is why this case provide very good results with the instance 6 (very close to the best case on average).

If $\lambda = 0.999$ (case 30), the final temperature is 0.045 which is just near zero so the exploitation and exploration is finished just after 10 000 evaluations.



If $\lambda = 0.9999$ (case 31), then the final temperature is around 368 so the algorithm still accept worse solution so the exploitation is not yet finished.

		Simulated Annealing [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	28	29	30	31	28	29	30	31	28	29	30	31
Score	best*	14217.0	14208.0	14049.0	14738.0	21299.0	21117.0	21208.0	21897.0	34073.0	34210.0	34007.0	34946.0
	worst	14695.0	14665.0	14345.0	15154.0	22252.0	22585.0	21980.0	22762.0	34816.0	34728.0	34631.0	35627.0
	avg	14376.1	14383.9	14208.3	14941.9	21694.5	21768.7	21481.3	22370.3	34508.2	34475.4	34251.9	35265.7
	std	160.95	134.78	101.67	140.61	317.73	475.87	287.93	264.43	211.71	205.53	162.76	183.97
Time	second (s)	8.425	8.436	8.436	8.652	17.802	17.621	17.504	17.066	33.683	33.699	33.879	34.127
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 35: Results for each case and each basic instance to get the best cooling rate for the simulated annealing.

Furthermore, we can observe on the basic instances [Table 35](#) and the medium instances [Table 36](#) that the case 30 is the best on average for instance 1, 2, 3 and 4.

		Simulated Annealing [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	28	29	30	31	28	29	30	31
Score	best*	315515.0	311141.0	313203.0	324766.0	386730.0	387407.0	388299.0	395165.0
	worst	319258.0	320268.0	316507.0	330087.0	392984.0	394815.0	392236.0	402408.0
	avg	316743.4	315174.1	315110.5	328330.3	389859.9	390010.8	390305.8	399671.0
	std	1327.58	2533.88	1269.85	1544.48	1914.81	2520.44	1404.08	2079.87
Time	second (s)	83.402	83.018	84.306	83.741	169.287	171.097	171.812	172.504
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 36: Results for each case and each medium instance to get the best cooling rate for the simulated annealing.

Lastly, on the hard instance ([Table 37](#)), the case 30 is very close to the best one average (case 31) so we can finally conclude that the best overall is the case 30. Thus, we take a cooling rate of $\lambda = 0.999$.

		Simulated Annealing [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	28	29	30	31
Score	best*	7191462.0	7164601.0	7183719.0	7148733.0
	worst	7242522.0	7234953.0	7277444.0	7243627.0
	avg	7218795.0	7198379.1	7215244.5	7190556.5
	std	18751.32	18516.45	29218.04	29526.12
Time	second (s)	879.441	882.499	879.332	859.753
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 37: Results for each case on the hard instance to get the best cooling rate for the simulated annealing.

4.3 Summary of the parameters

5 Solving the permutation flowshop scheduling problem

5.1 Comparison of all methods on every instances

5.2 Convergence of Evolutionary Algorithm

5.3 Best sequences found

Conclusion

References

- [1] Stack Overflow contributors. *python - time.time vs. timeit.timeit*. <https://stackoverflow.com/questions/17579357/time-time-vs-timeit-timeit>. See on 2026/03/29. 2013.
- [2] Michal Przewozniczek et al. *Task 0: Implementation and Study of Evolutionary Algorithms Compared to Non-evolutionary Methods*. Course Assignment, Wrocław University of Science and Technology (PWr). Project description. Mar. 2026.



Appendixes

List of Tables

1	Hardware specifications of the machine used for computation.	5
2	Description of the parameters of the cases used to determine the best population/- generation ratio.	9
3	Description of the parameters of the cases used to determine the best crossover probability.	9
4	Description of the parameters of the cases used to determine the best mutation probability.	10
5	Description of the parameters of the cases used to determine the best tour size. . .	10
6	Description of the parameters of the cases used to determine the best mutation type. Comparison between swap and inversion mutation.	10
7	Description of the parameters of the cases used to determine the best crossover type.	11
8	Description of the parameters of the cases used to determine the impact of elitism.	11
9	Description of the parameters used to determine the best initial temperature for Simulated Annealing.	11
10	Description of the parameters used to determine the best cooling rate for Simu- lated Annealing.	12
11	Results for each case and each basic instance to get the best ratio generation/pop- ulation size for the evolutionary algorithm.	13
12	Results for each case and each medium instance to get the best ratio genera- tion/population size for the evolutionary algorithm.	13
13	Results for each case on the hard instance to get the best ratio generation/popu- lation size for the evolutionary algorithm.	14
14	Results for each case and each basic instance to get the best crossover probability for the evolutionary algorithm.	14
15	Results for each case and each medium instance to get the best crossover proba- bility for the evolutionary algorithm.	15
16	Results for each case on the hard instance to get the best crossover probability for the evolutionary algorithm.	15
17	Results for each case and each basic instance to get the best mutation probability for the evolutionary algorithm.	16
18	Results for each case and each medium instance to get the best mutation proba- bility for the evolutionary algorithm.	16
19	Results for each case on the hard instance to get the best mutation probability for the evolutionary algorithm.	17
20	Results for each case and each basic instance to get the best tour size for the evo- lutionary algorithm.	17



21	Results for each case and each medium instance to get the best tour size for the evolutionary algorithm.	18
22	Results for each case on the hard instance to get the best tour size for the evolutionary algorithm.	18
23	Results for each case and each basic instance to get the best mutation type for the evolutionary algorithm.	19
24	Results for each case and each medium instance to get the best mutation type for the evolutionary algorithm.	19
25	Results for each case on the hard instance to get the best mutation type for the evolutionary algorithm.	20
26	Results for each case and each basic instance to get the best crossover type for the evolutionary algorithm. The color yellow indicates that we reach the best known value for the case 19 on the instance 3.	20
27	Results for each case and each medium instance to get the best crossover type for the evolutionary algorithm.	21
28	Results for each case on the hard instance to get the best crossover type for the evolutionary algorithm.	21
29	Results for each case and each basic instance to study the impact of elitism for the evolutionary algorithm.	22
30	Results for each case and each medium instance to study the impact of elitism for the evolutionary algorithm.	22
31	Results for each case on the hard instance to study the impact of elitism for the evolutionary algorithm.	23
32	Results for each case and each basic instance to get the best initial temperature for the simulated annealing.	23
33	Results for each case and each medium instance to get the best initial temperature for the simulated annealing.	24
34	Results for each case on the hard instance to get the best initial temperature for the simulated annealing.	24
35	Results for each case and each basic instance to get the best cooling rate for the simulated annealing.	25
36	Results for each case and each medium instance to get the best cooling rate for the simulated annealing.	25
37	Results for each case on the hard instance to get the best cooling rate for the simulated annealing.	26

List of Figures

1	Class diagram of the application implemented.	7
---	---	---

List of Algorithms



Listings

1	Allocating core 1 to run the experiment	5
2	Control the number of cores used in the <code>main</code> file of our application	5
3	Example of unit test for the evaluation function of a given problem	8
4	Implementation of the mutation	16